Input/Output (I/O)

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 Introduction

This lecture examines how Linux represents, accesses, and manages files in practice. Through a series of experiments and programming exercises, we will study file metadata and inodes, random file access with `lseek()`, memory-mapped files using `mmap()`, directory traversal, file locking, and file-system mounting.

2 Inode Metadata and stat()

Every file stored in a Linux file system is represented internally by an **inode** (index node). An inode stores metadata about a file, including its size, ownership, permissions, timestamps, and the locations of its data blocks on disk. The file name itself is not stored in the inode. Instead, directories map file names to inode numbers.

In this exercise, we will create a file, inspect its inode information using Linux commands, and then retrieve the same metadata programmatically using the `stat()` system call.

2.1 Creating a Test File

Create a simple text file:

```
echo "Operating Systems" > file.txt
```

2.2 Inspecting File Metadata

To display the inode number together with the file information, execute:

```
ls -li file.txt
```

Example output:

```
7892758 -rw-rw-r-- 1 sareh sareh 18 Jun  8 10:56 file.txt
```

The first column represents the inode number assigned to the file by the file system. To examine the metadata stored in the inode in more detail, execute the `stat` command as shown below. This command retrieves the file's metadata from the file system and displays it in a human-readable format.

```
stat file.txt
```

Example output:

```
  File: file.txt
  Size: 18              Blocks: 8          IO Block: 4096   regular file
Device: 259,7   Inode: 7892758     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   sareh)   Gid: ( 1000/   sareh)
Access: 2026-06-08 10:56:19.008870268 +0200
Modify: 2026-06-08 10:56:19.008870268 +0200
Change: 2026-06-08 10:56:19.008870268 +0200
 Birth: 2026-06-08 10:56:19.008870268 +0200
```

2.3 Obtaining File Metadata Using stat()

Linux provides the `stat()` system call to retrieve file metadata programmatically. Consider the following program `01_stat.c`:

```c
#include <stdio.h>
#include <sys/stat.h>

int main() {
    struct stat st;
    if (stat("../file.txt", &st) == -1) {
        perror("stat");
        return 1;
    }

    printf("Inode Number : %lu\n", st.st_ino);
    printf("File Size    : %ld bytes\n", st.st_size);
    printf("Link Count   : %ld\n", st.st_nlink);

    return 0;
}
```

Example output:

```
Inode Number : 616918
File Size    : 18 bytes
Link Count   : 1
```

> `stat()` retrieves metadata about a file and stores it in the struct `st`. It returns 0 if it was successful and -1 if an error occurs.

2.4 Important Fields of struct stat

- `st_ino`: inode number
- `st_size`: file size in bytes
- `st_mode`: file type and permission bits
- `st_nlink`: number of hard links
- `st_uid`: owner user ID
- `st_gid`: owner group ID
- `st_atime`: last access time
- `st_mtime`: last modification time
- `st_ctime`: last metadata change time

2.5 Creating and Observing Hard Links

Create a file and a hard link:

```
ln file.txt file_link.txt
ls -li
```

Example output:

```
616918 -rw-rw-r-- 2 sareh sareh  18 Jun  8 11:11 file_link.txt
616918 -rw-rw-r-- 2 sareh sareh  18 Jun  8 11:11 file.txt
```

Notice that both names reference the same inode. The link count is now 2, indicating that two directory entries point to the same file object.
You can verify this using:

```
stat file.txt
stat file_link.txt
```

2.6 Open Files and File Deletion

Linux keeps track of open files through file descriptors. A file can remain accessible even after its directory entry has been removed. This behavior can be observed using the `lsof` command. Create a file:

```
echo "Operating Systems" > temp.txt
```

Consider the following program `02_open_file.c`:

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd = open("../temp.txt", O_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    printf("PID: %d\n", getpid());
    getchar();
    char buf[64];
    int n = read(fd, buf, sizeof(buf)-1);
    buf[n] = '\0';
    printf("Content: %s\n", buf);
    close(fd);
    return 0;
}
```

Execute the program and leave it waiting at `getchar()`. In another terminal, inspect the open file using:

```
lsof -p <pid>
```

Example output:

```
COMMAND    PID    USER   FD   TYPE  DEVICE  SIZE/OFF  NODE    NAME
02_open_f  18004  sareh  3r   REG   259,4   18        616921  .../temp.txt
```

The `lsof` command displays files currently opened by a process. Now remove the file:

```
rm temp.txt
```

Notice that the file no longer appears in the directory. However, if you execute `lsof -p <pid>`, you may observe:

```
COMMAND    PID    USER   FD   TYPE  DEVICE  SIZE/OFF  NODE    NAME
02_open_f  18004  sareh  3r   REG   259,4   18        616921  .../temp.txt (deleted)
```

The file has been removed from the directory, but the process still holds an open file descriptor that references the inode. **Return to the program and press Enter**. The program successfully reads:

```
Content: Operating Systems
```

This demonstrates that removing a file deletes only the directory entry. The inode and data blocks remain allocated as long as at least one process holds an open file descriptor or another hard link references the inode. The storage is finally released only when the last reference disappears.

3 Direct Random Access with lseek()

When a file is opened, the operating system maintains a current file offset that determines where the next read or write operation will occur. Normally, this offset advances automatically as data is accessed. The `lseek()` system call allows a program to move the file offset to a desired location, making it possible to read from or write to any position within a file without processing all preceding data.
First, create a text file containing several lines:

```
echo "Line 1" > file.txt
echo "Line 2" >> file.txt
echo "Line 3" >> file.txt
echo "Line 4" >> file.txt
```

3.1 Reading from an Arbitrary Position

The `lseek()` system call changes the current file offset associated with an open file descriptor. The prototype is:

```
off_t lseek(int fd, off_t offset, int whence);
```

The parameter `whence` determines how the offset is interpreted:

- `SEEK_SET`: relative to the beginning of the file
- `SEEK_CUR`: relative to the current position
- `SEEK_END`: relative to the end of the file

3.2 Reading from an Arbitrary Position

Consider the following program `03_lseek.c`:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("../file.txt", O_RDONLY);
    printf("Current offset: %ld\n",
           lseek(fd, 0, SEEK_CUR));
    lseek(fd, 7, SEEK_SET);
    printf("Current offset: %ld\n",
           lseek(fd, 0, SEEK_CUR));
    char buf[8];
    ssize_t n = read(fd, buf, 6);
    if (n == -1) {
        perror("read");
        close(fd);
        return 1;
    }
    buf[n] = '\0';
    printf("Read data: %s\n", buf);
    close(fd);
    return 0;
}
```

The first call `lseek(fd, 0, SEEK_CUR)` does not change the file position and is commonly used to obtain the current file offset. The call The first call `lseek(fd, 7, SEEK_SET)` moves the file offset to byte 7 relative to the beginning of the file. The subsequent `read()` operation starts reading from that location.
Example output:

```
Current offset: 0
Current offset: 7
Read data: Line 2
```

4 Memory-Mapped Files

Normally, a program accesses a file through the `read()` and `write()` system calls. In this approach, data is copied between kernel and user-space buffers during each I/O operation. Linux also provides an alternative mechanism called **memory mapping**, where a file is mapped directly into the virtual address space of a process. Once mapped, the file contents can be accessed using ordinary memory operations. The `mmap()` system call creates such a mapping between a file and a region of virtual memory.
Create a test file:

```
echo "Line 1" > file.txt
```

Then consider the following program `04_mmap.c`:

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>

int main() {

    int fd = open("../file.txt", O_RDWR);
    struct stat st;
    stat("file.txt", &st);

    char *data = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, fd
        , 0);

    if (data == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    printf("File content: %s\n", data);
    printf("PID: %d\n", getpid());
    getchar();
    data[0] = 'X';
    munmap(data, st.st_size);
    close(fd);
    return 0;
}
```

The `mmap()` system call maps the file into the process address space and returns a pointer to the mapped region. The file contents can then be accessed directly through memory operations.

Execute the program and leave it running. In another terminal, inspect the process memory mappings `cat /proc/<pid>/maps` or `pmap <pid>`. Example output:

```
7d8e01200000-7d8e03a00000 rw-s 00000000 103:04 616918 .../file.txt
```

Notice that `file.txt` appears as a memory region within the process address space.

Return to the program and press Enter. The statement `data[0] = 'X';` modifies the mapped memory region. Because the mapping was created using `MAP_SHARED`, the modification is propagated to the underlying file.

Verify the change with `cat file.txt`:

```
Xine 1
```

5 Directory Traversal

Directories in Linux are special files that contain entries mapping file names to inodes. To access the contents of a directory, Linux provides the functions `opendir()`, `readdir()`, and `closedir()`. These functions allow a program to iterate through all entries stored in a directory. Create a directory containing a few files:

```
mkdir testdir
touch testdir/file1.txt
touch testdir/file2.txt
touch testdir/file3.txt
ls testdir
```

Consider the following program `05_readdir_stat.c`:

```c
#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>

int main() {

    DIR *dir;
    struct dirent *entry;
    struct stat st;
    char path[1024];
    dir = opendir("../testdir");
    if (dir == NULL) {
        perror("opendir");
        return 1;
    }
    while ((entry = readdir(dir)) != NULL) {
        int len = snprintf(path, sizeof(path), "../testdir/%s", entry->d_name)↩
            ;
        if (len < 0 || len >= (int)sizeof(path)) {
            fprintf(stderr, "Path too long: %s\n", entry->d_name);
            continue;
        }
        if (stat(path, &st) == 0) {
            printf("Name : %s\n", entry->d_name);
            printf("Inode: %lu\n", (unsigned long) st.st_ino);
            printf("Size : %ld bytes\n\n", st.st_size);
        }
```

```
27      }
28      closedir(dir);
29      return 0;
30  }
```

Example output:

```
Name : .
Inode: 616930
Size : 0 bytes

Name : ..
Inode: 616913
Size : 4096 bytes

Name : file1.txt
Inode: 616931
Size : 0 bytes

Name : file2.txt
Inode: 616932
Size : 0 bytes

Name : file3.txt
Inode: 616933
Size : 0 bytes
```

The function `opendir()` opens a directory stream, and `readdir()` returns one directory entry at a time. Each directory entry contains a file name and associated inode information. To obtain additional metadata such as file size, the program constructs the full file path and calls `stat()` for each entry.

Notice the special entries `.` and `..`, which represent the current directory and its parent directory, respectively.

Directory traversal is a fundamental operation used by many Linux utilities, including `ls`, `find`, backup software, file-system scanners, and indexing tools.

6 File Locking

When multiple processes access the same file concurrently, race conditions may occur. For example, two processes writing to the same file at the same time may overwrite each other's data or produce inconsistent output. Linux provides the `flock()` system call to coordinate file access between processes by placing locks on open files.

6.1 Writing to a Shared File Without Locking (`06_no_lock.c`)

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5      FILE *fp = fopen("../log.txt", "a");
```

```
6     for (int i = 0; i < 5; i++) {
7         fprintf(fp, "PID %d: message %d\n", getpid(), i);
8         fflush(fp);
9         sleep(1);
10    }
11    fclose(fp);
12    return 0;
13 }
```

Compile and execute the program in two different terminals simultaneously, after both processes finish, inspect the file:

```
PID 30401: message 0
PID 30403: message 0
PID 30401: message 1
PID 30403: message 1
PID 30401: message 2
PID 30403: message 2
PID 30401: message 3
PID 30403: message 3
...
```

Notice that messages from both processes are interleaved. In more complex applications, concurrent writes may lead to corrupted or inconsistent data.

6.2 Using flock() to Synchronize Access (07_flock.c)

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <fcntl.h>
4  #include <sys/file.h>
5
6  int main() {
7
8      int fd = open("../log.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);
9
10     printf("PID %d waiting for lock...\n", getpid());
11
12     flock(fd, LOCK_EX);
13
14     printf("PID %d acquired lock\n", getpid());
15
16     for (int i = 0; i < 5; i++) {
17         printf("PID %d: message %d\n", getpid(), i);
18         dprintf(fd,"PID %d: message %d\n", getpid(), i);
19         sleep(1);
20     }
21
22     flock(fd, LOCK_UN);
23     printf("PID %d released lock\n", getpid());
24     close(fd);
25     return 0;
```

```
26 }
```

Compile and execute the program in two different terminals. The second process remains blocked until the first process releases the lock. As a result, only one process writes to the file at a time.
Inspect the resulting file:

```
PID 33702: message 0
PID 33702: message 1
PID 33702: message 2
PID 33702: message 3
PID 33702: message 4
PID 33704: message 0
PID 33704: message 1
PID 33704: message 2
PID 33704: message 3
PID 33704: message 4
```

6.3 Types of flock() Locks

The `flock()` system call supports several lock types:

- `LOCK_SH`: shared lock (multiple readers allowed)
- `LOCK_EX`: exclusive lock (single writer)
- `LOCK_UN`: release a lock
- `LOCK_NB`: non-blocking lock request

For example:

```
1 flock(fd, LOCK_EX | LOCK_NB);
```

attempts to acquire an exclusive lock immediately. If another process already holds the lock, the call fails instead of waiting.

7 Mounting File Systems

Linux presents all files and directories through a single hierarchical directory tree rooted at `/`. To make the contents of a storage device accessible, its file system must be attached to this directory tree. This process is called **mounting**. The directory where a file system is attached is called a **mount point**.

Unlike some operating systems that assign separate drive letters to storage devices, Linux integrates all file systems into a unified namespace.

7.1 Inspecting Mounted File Systems

To display all currently mounted file systems, execute:

```
mount
```

The output may be extensive. A more readable view can be obtained with:

```
df -h
```

Example output:

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   18G   30G  38% /
tmpfs           1.9G     0  1.9G   0% /dev/shm
/dev/sdb1       100G   25G   70G  27% /data
```

The last column shows the mount point where each file system is attached.

7.2 Examining the Mount Table

Linux maintains information about mounted file systems in:

```
cat /proc/mounts
```

Example output:

```
/dev/sda1 / ext4 rw,relatime 0 0
tmpfs /dev/shm tmpfs rw,nosuid,nodev 0 0
/dev/sdb1 /data ext4 rw,relatime 0 0
```

Each entry contains the device name, mount point, file system type, and mount options. For example, the entry:

```
/dev/sdb1 /data ext4 rw,relatime 0 0
```

indicates that the device `/dev/sdb1` is mounted at `/data` using the `ext4` file system.

7.3 Viewing Storage Devices

To view available storage devices and partitions, execute:

```
lsblk
```

Example output:

```
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda      8:0    0   50G  0 disk
|-sda1   8:1    0   50G  0 part /
sdb      8:16   0  100G  0 disk
|-sdb1   8:17   0  100G  0 part /data
```

This command shows the relationship between physical devices, partitions, and mount points.

7.4 Temporary Mounting Using tmpfs

Linux supports several file-system types. One useful example is `tmpfs`, a memory-backed file system whose contents reside in RAM.
Create a mount point:

```
sudo mkdir /mnt/mytmp
```

Mount a temporary file system:

```
sudo mount -t tmpfs tmpfs /mnt/mytmp
```

Verify the mount:

```
df -h /mnt/mytmp
```

Create a file inside the mounted file system:

```
echo "Operating Systems" > /mnt/mytmp/test.txt
cat /mnt/mytmp/test.txt
```

Example output:

```
Operating Systems
```

Now unmount the file system:

```
sudo umount /mnt/mytmp
```

The mounted file system disappears from the directory tree, and its contents are no longer accessible.

7.5 Virtual File Systems

Not all mounted file systems correspond to physical disks. Linux also uses virtual file systems that provide kernel information through files.
Examples include:

- `/proc` for process and kernel information,
- `/sys` for device and hardware information,
- `/dev` for device files,
- `tmpfs` for temporary memory-backed storage.

You can observe these file systems using:

```
mount | grep -E "proc|sysfs|tmpfs"
```